

```
# =====  
# PREPROCESSING IMPACT ON EDGE DETECTION  
# =====  
  
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
from google.colab import files  
  
# -----  
# 1. Upload Image  
# -----  
uploaded = files.upload()  
  
image_name = list(uploaded.keys())[0]  
  
# Read image  
img = cv2.imread(image_name)  
  
# Convert BGR to RGB for displaying  
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)  
  
# Convert to grayscale  
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)  
  
# -----  
# 2. Edge Detection on RAW IMAGE  
# -----  
raw_edges = cv2.Canny(gray, 50, 150)  
  
# -----  
# 3. NOISE REDUCTION  
# -----  
# Gaussian filtering reduces random noise  
blurred = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
# -----
# 4. UNEVEN ILLUMINATION CORRECTION
# -----
# CLAHE improves local contrast
clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

enhanced = clahe.apply(blurred)

# -----
# 5. EDGE DETECTION AFTER PREPROCESSING
# -----
processed_edges = cv2.Canny(
    enhanced,
    50,
    150
)

# -----
# 6. MORPHOLOGICAL PROCESSING
# -----
# Kernel used to connect small gaps
kernel = np.ones((3, 3), np.uint8)

# Closing connects fragmented edges
continuous_edges = cv2.morphologyEx(
    processed_edges,
    cv2.MORPH_CLOSE,
    kernel,
    iterations=1
)

# -----
# 7. DISPLAY RESULTS
# -----
plt.figure(figsize=(15, 10))
```

```
plt.figure(figsize=(15, 10))

plt.subplot(2, 3, 1)
plt.imshow(img_rgb)
plt.title("1. Original Image")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(gray, cmap="gray")
plt.title("2. Grayscale Image")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(raw_edges, cmap="gray")
plt.title("3. Canny on Raw Image")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(enhanced, cmap="gray")
plt.title("4. After Preprocessing")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(processed_edges, cmap="gray")
plt.title("5. Canny After Preprocessing")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(continuous_edges, cmap="gray")
plt.title("6. Continuous Edges")
plt.axis("off")

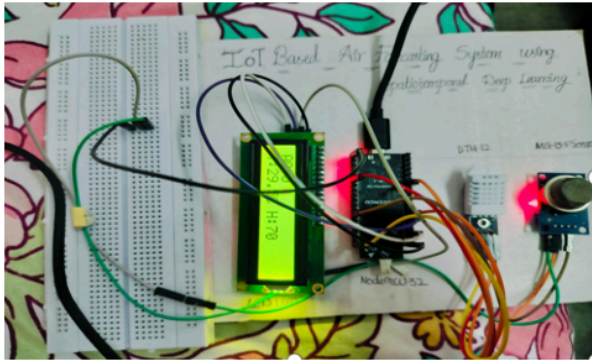
plt.tight_layout()
plt.show()
```

Choose Files Screenshot... 204629.png

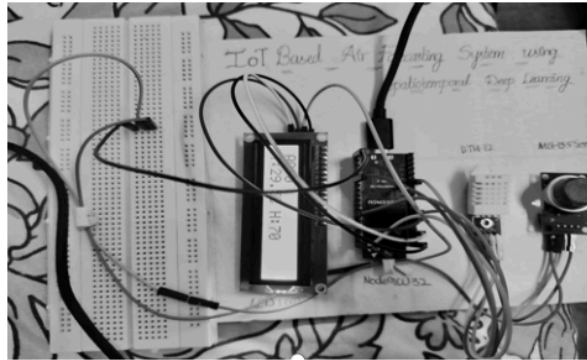
Screenshot 2026-06-24 204629.png(image/png) - 312424 bytes, last modified: 6/24/2026 - 100% done

Saving Screenshot 2026-06-24 204629.png to Screenshot 2026-06-24 204629.png

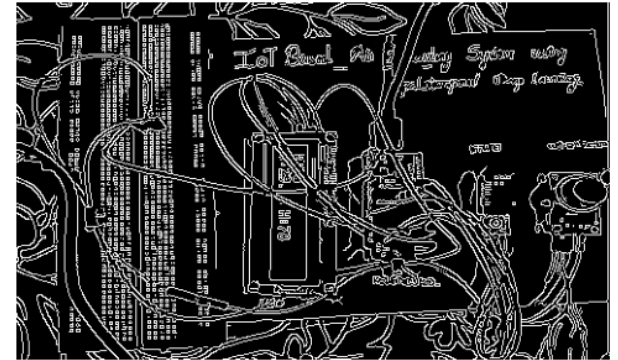
1. Original Image



2. Grayscale Image



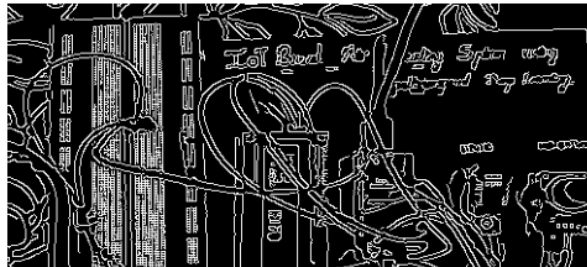
3. Canny on Raw Image



4. After Preprocessing



5. Canny After Preprocessing



6. Continuous Edges



